

Milind Bansal

📞 +91-XXXX-XXXXXXX | ✉ milindsk8r@gmail.com | 🔗 linkedin.com/in/milind-bansal | 🐙 github.com/MilindDevX

EDUCATION

Your University Name

Bachelor of Technology / Science in Computer Science

City, Country

Expected Month YYYY

- GPA: X.X/10 • Relevant Coursework: Data Structures, Algorithms, Databases, Operating Systems, Web Development

TECHNICAL SKILLS

Languages: TypeScript, JavaScript (ES2024), Python, SQL

Frontend: React 19, Next.js 14/16 (App Router), Vite, Tailwind CSS, Framer Motion, Recharts, Zustand

Backend: Node.js, Express.js 5, tRPC, REST API Design, JWT Authentication, Zod Validation

Databases: PostgreSQL, Prisma ORM, NeonDB (serverless), Redis

Infrastructure: Docker, Docker Compose, Vercel, Render, BullMQ (job queues), Cloudinary

Testing: Vitest, Supertest, Jest, Playwright (E2E)

Tools: Git, GitHub, OpenAPI/Swagger, Postman, ESLint, Biome

EXPERIENCE

Full-Stack Developer (Freelance)

Remote

Digital Heroes

2024 – Present

- Designed and shipped **RouteLens**, a zero-backend API management tool used for parsing OpenAPI 2.0/3.x specs and diffing JSON responses, eliminating the need for any server infrastructure and reducing developer setup time to zero.
- Engineered a structural JSON diff algorithm with identity-key array matching (by `id/name/slug`), preventing false-positive diffs on reordered arrays and improving diff accuracy significantly over naive line-diff tools.
- Implemented a quota-safe **localStorage** persistence layer that gracefully degrades in incognito mode, ensuring zero data-loss errors across all browser environments.

PROJECTS

MedMarket India | *Node.js, Express 5, TypeScript, React 19, PostgreSQL, Prisma, BullMQ, Docker* 🐙 🔗 API Docs 2024 – 2025

- [S] Large-scale multi-sided healthcare marketplace required simultaneous support for consumer, pharmacy, and admin workflows under strict CDSCO/DPCO regulatory requirements; [T] architected a monorepo with role-based Express 5 REST API and Prisma ORM; [A] designed 3 distinct React portals with 30+ protected routes using JWT (15-min access + 7-day refresh rotation) and Zod-validated request schemas; [R] delivered a production-deployed platform on Render + Vercel serving all 3 user roles with **58 passing tests** (38 backend + 20 frontend).
- Implemented a **BullMQ + Redis nightly expiry queue** to proactively alert pharmacies of inventory expiring within 60/30/45 days, reducing dead-stock risk and replacing what would have been a blocking cron job.
- Built a **pharmacy analytics dashboard** (Recharts) delivering 14-day revenue trends, hourly order heatmaps, fulfillment rates, and dead-stock value metrics—providing pharmacy owners with actionable daily insights without raw database access.
- Reduced API response time for analytics endpoints via **parallel Prisma aggregation queries** (Promise.all), cutting page-load wait time compared to sequential queries.

FeedbackOS | *Next.js 16, TypeScript, BullMQ, Redis, PostgreSQL, Prisma, Groq API, Vercel* 🐙 🔗 Live 2025

- [S] Product managers spent 5–10 hours/week manually tagging customer feedback from Zendesk, Intercom, and CSV sources; [T] built an automated triage system with AI-powered classification; [A] implemented a **two-process architecture** (Next.js HTTP + BullMQ background worker) so AI inference never blocks API response times, with AES-256-GCM encryption for integration credentials; [R] reduced classification latency to **~1.2s end-to-end** and eliminated manual tagging entirely.
- Integrated **Zendesk API, Intercom webhooks, and CSV drag-and-drop** (Papaparse, up to 500 rows/5MB) ingestion pipelines, enabling multi-channel feedback consolidation in a single dashboard.
- Implemented a **5-minute Redis cache layer** on Postgres dashboard aggregations, protecting the database from repeated expensive queries under concurrent user load.

RouteLens (apidiff) | *Next.js 14 (Static Export), TypeScript, Tailwind CSS, Vercel* 🐙 🔗 Live 2024

- Built a **100% client-side, zero-backend** developer tool that parses OpenAPI 2.0/3.x specs (JSON/YAML) with automatic **\$ref** resolution, eliminating the need for Swagger UI server setup.
- Engineered a **structural JSON diff viewer** that matches array objects by identity keys (`id, name, slug`) rather than index, preventing false-positive change detection on reordered API responses.

QuickNotes | *React, NeonDB (Serverless PostgreSQL)* 🐙 2024

- Developed a fast, responsive note-taking application with serverless Postgres (NeonDB) for instant cold-start connection pooling, ensuring sub-100ms database round-trips without dedicated server infrastructure.

CERTIFICATIONS & ACHIEVEMENTS

Digital Heroes Programme – Certificate of Completion (2024)

Open Source Contributions: RouteLens deployed and maintained for Digital Heroes team